

CST8513: Quality Assurance and Testing

Assignment III (Phase 3: Automation Testing)

Deliverable 1: Automated Test Scripts with Page Object Model (POM) Architecture

Target Application: SauceDemo (Swag Labs) - <https://www.saucedemo.com/>

Students: Devangbhai Pandit & Meet Ahalpara

Professor: Prof. Sharmista Datta

Course: CST8513 – Quality Assurance and Testing

Institution: Algonquin College – School of Advanced Technology

Date: August 2026 | Academic Term: Summer 2026

GitHub Repository: <https://github.com/MeetAhalpara/QA-Test-Automation>

Table of Contents

CST8513: Quality Assurance and Testing	1
Assignment III (Phase 3: Automation Testing)	1
Deliverable 1: Automated Test Scripts with Page Object Model (POM) Architecture	1
1. Executive Overview & Architecture	3
1.1 Page Object Model (POM) Design Pattern	3
2. Framework File Structure & Code References	4
3. Test Traceability Matrix (TC-01 through TC-14)	6
4. Configuration & Foundation Code	9
4.1 Configuration Module (automation_selenium/config/config.py)	9
4.2 Base Page Object (automation_selenium/pages/base_page.py)	10
5. Page Object Implementation Classes	13
5.1 Login Page Object (automation_selenium/pages/login_page.py)	13
5.2 Inventory Page Object (automation_selenium/pages/inventory_page.py)	15
6. Automated Test Suites (Assertions & Execution)	18
6.1 Authentication Test Suite (automation_selenium/tests/test_authentication.py)	18
6.2 Checkout & Tax Calculations Suite (automation_selenium/tests/test_checkout.py)	20
7. Execution Commands & Technical References	23
7.1 Local Test Execution	23
References	23

1. Executive Overview & Architecture

This document constitutes Deliverable 1 for CST8513 Assignment III. It presents the complete test automation codebase implemented for the SauceDemo web application using Python 3.11, Selenium WebDriver, Pytest, and Robot Framework.

1.1 Page Object Model (POM) Design Pattern

The framework strictly adheres to the Page Object Model (POM) design pattern. POM is an industry-standard architectural pattern that creates an object repository for web UI elements. The advantages realized in this project include:

1. Separation of Concerns: Element locators and UI interaction mechanisms are strictly isolated within Page Classes. Test files contain exclusively high-level business actions and validation assertions.
2. Maintainability: If an HTML element locator (e.g., ID, CSS, XPath) changes on SauceDemo, updates are made in a single Page Object file without modifying any test scripts.
3. Reusability: Common actions such as user login, adding products, and navigating to checkout are defined once and reused across multiple test suites.
4. Robust Synchronization: All UI interactions utilize explicit waits (`WebDriverWait`) and dynamic expected conditions, eliminating arbitrary sleeps and preventing execution flakiness.

2. Framework File Structure & Code References

Assignment3/ ├── automation_selenium/ k ├── config/ └── config.py ls, credentials, paths ├── pages/ ├── base_page.py s, clicks, React sync) └── login_page.py 02) └── inventory_page.py (FR-03, FR-04, FR-06) └── product_details_page.py (FR-05) └── cart_page.py R-07, FR-08) └── checkout_step_one_page.py R-09, FR-10) └── checkout_step_two_page.py d total (FR-11) └── checkout_complete_page.py FR-13) └── tests/ s ├── conftest.py ilure screenshot hooks ├── test_authentication.py s), TC-14 (Logout) ├── test_inventory.py ia), TC-05 (Details) ├── test_cart.py perations) ├── test_checkout.py TC-13 (Checkout flow) └── test_e2e_workflow.py ifecycle ├── automation_robot/ ├── resources/ └── variables.resource ata └── common.resource s └── tests/ └── saucedemo_suite.robot to TC-14 ├── ci_cd/ └── Jenkinsfile	# Selenium WebDriver Framework # Configuration constants, UR # Page Object Classes # Master parent wrapper (wait # Login interface (FR-01, FR- # Product catalog & sorting # Product specifications view # Shopping cart management (F # Customer info input form (F # Overview, subtotal, tax, an # Order confirmation (FR-12, # Automated Pytest Test Suite # Fixtures, browser setup, fa # TC-01, TC-02 (4 combination # TC-03, TC-04 (4 sort criter # TC-06, TC-07, TC-08 (Cart o # TC-09, TC-10, TC-11, TC-12, # Full End-to-End purchase li # Robot Framework Test Suite # Locators, variables, test d # Reusable high-level keyword # Automated test cases TC-01 # Continuous Integration # Jenkins CI declarative pipe
---	---

```

line
├── .github/workflows/automation-tests.yml # GitHub Actions CI workflow
├── reports/
│   ├── selenium_test_report.html # Interactive visual Pytest H
TML report
│   ├── junit_selenium.xml # JUnit XML report for CI tes
t parsing
│   ├── robot_logs/ # Robot Framework report.html
│   └── & log.html
├── run_tests.py # Unified CLI test runner
├── pytest.ini # Pytest configuration & mark
ers
└── requirements.txt # Python dependencies

```

3. Test Traceability Matrix (TC-01 through TC-14)

Test ID	Functional Requirement	Feature Area	Automated Test Method	Verification Summary
TC-01	FR-01	Authentication	test_tc01_successful_login	Validates standard_user login, URL redirection to /inventory.html, and catalog rendering.
TC-02	FR-02	Authentication	test_tc02_login_error_validation	Parameterized test checking invalid, locked_out, empty username, and empty password errors.
TC-03	FR-03	Inventory Display	test_tc03_inventory_display	Asserts exact count of 6 products with titles, prices, descriptions, and images.
TC-04	FR-04	Product Sorting	test_tc04_product_sorting	Validates all 4 sort options: Name (A-Z), Name (Z-A), Price (Low-High), Price (High-Low).
TC-05	FR-05	Product Details	test_tc05_product_details_view	Validates navigation to item details, \$29.99 price display, image, and back button.
TC-06	FR-06	Shopping Cart	test_tc06_add_to_cart	Validates adding items from inventory

Test ID	Functional Requirement	Feature Area	Automated Test Method	Verification Summary
				and details pages increments cart badge dynamically.
TC-07	FR-07	Shopping Cart	test_tc07_remove_from_cart	Validates removing items decrements badge count and updates button state.
TC-08	FR-08	Cart Review	test_tc08_cart_review	Validates /cart.html item list, price accuracy, and continue shopping navigation.
TC-09	FR-09	Checkout Information	test_tc09_checkout_empty_fields_validation	Submitting empty form blocks checkout with "Error: First Name is required".
TC-10	FR-10	Input Validation	test_tc10_checkout_input_validation	Parameterized validation for missing Last Name and missing Postal Code errors.
TC-11	FR-11	Order Calculations	test_tc11_order_overview_calculations	Mathematical formula check: Subtotal (\$45.98) + 8% Tax (\$3.68) == Total (\$49.66).
TC-12	FR-12	Order Completion	test_tc12_finish_order_confirmation	Validates "Thank you for your order!"

Test ID	Functional Requirement	Feature Area	Automated Test Method	Verification Summary
TC-13	FR-13	Checkout Cancel	test_tc13_cancel_checkout_redirection	confirmation header and pony express graphic. Validates cancel from Step 1 returns to Cart; cancel from Step 2 returns to Inventory.
TC-14	FR-14	User Logout	test_tc14_secure_logout	Validates sidebar logout, session termination, and unauthorized back-navigation block.
E2E	Full Flow	E-Commerce Journey	test_complete_e2e_shopping_workflow	13-step comprehensive purchase lifecycle from login to order confirmation to logout.

4. Configuration & Foundation Code

4.1 Configuration Module (automation_selenium/config/config.py)

```
"""
Configuration module for SauceDemo Selenium Test Automation Framework.
Centralizes URLs, test credentials, explicit timeouts, and file paths.
Course: CST8513 - Quality Assurance and Testing (Assignment III)
Students: Devangbhai Pandit & Meet Ahalpara
"""

import os
from pathlib import Path

# Base Paths
PROJECT_ROOT = Path(__file__).resolve().parents[2]
REPORTS_DIR = os.path.join(PROJECT_ROOT, "reports")
SCREENSHOTS_DIR = os.path.join(REPORTS_DIR, "screenshots")

# Application Settings
BASE_URL = "https://www.saucedemo.com/"
DEFAULT_TIMEOUT = 10 # Seconds for explicit waits

# Test Credentials
VALID_USERNAME = "standard_user"
VALID_PASSWORD = "secret_sauce"
LOCKED_USERNAME = "locked_out_user"
PROBLEM_USERNAME = "problem_user"
PERFORMANCE_USERNAME = "performance_glitch_user"
INVALID_USERNAME = "invalid_user"
INVALID_PASSWORD = "wrong_password"

# Default Customer Information for Checkout
DEFAULT_FIRST_NAME = "Meet"
DEFAULT_LAST_NAME = "Ahalpara"
DEFAULT_POSTAL_CODE = "K2G 1V8"
```

4.2 Base Page Object (automation_selenium/pages/base_page.py)

```
"""
Base Page module implementing the Page Object Model (POM) foundation.
Provides common wrapper methods with explicit waits and React event dispatchi
ng for reliable UI interactions.
Course: CST8513 - Quality Assurance and Testing (Assignment III)
Students: Devangbhai Pandit & Meet Ahalpara
"""
```

```
import os
import time
from datetime import datetime
from selenium.webdriver.remote.webdriver import WebDriver
from selenium.webdriver.remote.webelement import WebElement
from selenium.webdriver.support.ui import WebDriverWait, Select
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import (
    TimeoutException,
    NoSuchElementException,
)
from automation_selenium.config.config import DEFAULT_TIMEOUT, SCREENSHOTS_DI
R
```

```
class BasePage:
    """Base class that all Page Objects inherit from."""

    def __init__(self, driver: WebDriver, timeout: int = DEFAULT_TIMEOUT):
        self.driver = driver
        self.timeout = timeout
        self.wait = WebDriverWait(self.driver, self.timeout)

    def navigate_to(self, url: str) -> None:
        """Navigates to the specified URL."""
        self.driver.get(url)

    def get_current_url(self) -> str:
        """Returns the current browser URL."""
        return self.driver.current_url

    def get_title(self) -> str:
        """Returns the current page title."""
        return self.driver.title

    def find_element(self, locator: tuple[str, str], timeout: int = None) ->
WebElement:
        """Waits for an element to be present in the DOM and returns it."""
        wait = self.wait if timeout is None else WebDriverWait(self.driver, t
```

```

imeout)
    return wait.until(EC.presence_of_element_located(locator))

    def find_visible_element(self, locator: tuple[str, str], timeout: int = None) -> WebElement:
        """Waits for an element to be visible on the page and returns it."""
        wait = self.wait if timeout is None else WebDriverWait(self.driver, timeout)
        imeout)
            return wait.until(EC.visibility_of_element_located(locator))

    def find_elements(self, locator: tuple[str, str], timeout: int = None) -> list[WebElement]:
        """Waits for elements to be present in the DOM and returns the list."""
        ""
        wait = self.wait if timeout is None else WebDriverWait(self.driver, timeout)
        imeout)
            try:
                wait.until(EC.presence_of_element_located(locator))
                return self.driver.find_elements(*locator)
            except TimeoutException:
                return []

    def click_element(self, locator: tuple[str, str], timeout: int = None) -> None:
        """Waits until element is present and clicks it via JavaScript for seamless React compatibility."""
        element = self.find_element(locator, timeout=timeout)
        self.driver.execute_script("arguments[0].click();", element)

    def enter_text(self, locator: tuple[str, str], text: str, clear_first: bool = True) -> None:
        """
        Enters text into an input field, ensuring React state synchronization.
        Uses React _valueTracker update and event dispatching for 100% reliable form state.
        """
        element = self.find_element(locator)
        script = """
        let input = arguments[0];
        let val = arguments[1];
        let lastValue = input.value;
        input.value = val;
        let tracker = input._valueTracker;
        if (tracker) {
            tracker.setValue(lastValue);
        }
        let inputEvent = new Event('input', { bubbles: true });
        input.dispatchEvent(inputEvent);
        let changeEvent = new Event('change', { bubbles: true });

```

```

        input.dispatchEvent(changeEvent);
        """
        self.driver.execute_script(script, element, text)

    def get_element_text(self, locator: tuple[str, str], timeout: int = None)
-> str:
        """Returns the text content of an element."""
        element = self.find_element(locator, timeout=timeout)
        return element.text.strip()

    def is_element_visible(self, locator: tuple[str, str], timeout: int = 3)
-> bool:
        """Checks if an element is visible within a short timeout without thr
owing."""
        try:
            wait = WebDriverWait(self.driver, timeout)
            wait.until(EC.visibility_of_element_located(locator))
            return True
        except (TimeoutException, NoSuchElementException):
            return False

    def select_dropdown_by_value(self, locator: tuple[str, str], value: str)
-> None:
        """Selects an option from a <select> element by its value attribute."
        """
        element = self.find_element(locator)
        select = Select(element)
        select.select_by_value(value)

    def take_screenshot(self, test_name: str = "screenshot") -> str:
        """Captures a PNG screenshot and saves it to the reports/screenshots
        directory."""
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        filename = f"{test_name}_{timestamp}.png"
        filepath = os.path.join(SCREENSHOTS_DIR, filename)
        self.driver.save_screenshot(filepath)
        return filepath

```

5. Page Object Implementation Classes

5.1 Login Page Object (automation_selenium/pages/login_page.py)

```
"""
LoginPage module representing the authentication interface of SauceDemo.
Course: CST8513 - Quality Assurance and Testing (Assignment III)
Students: Devangbhai Pandit & Meet Ahalpara
"""

import time
from selenium.webdriver.common.by import By
from selenium.webdriver.remote.webdriver import WebDriver
from automation_selenium.pages.base_page import BasePage
from automation_selenium.config.config import BASE_URL, VALID_USERNAME, VALID_PASSWORD


class LoginPage(BasePage):
    """Encapsulates locators and actions for the SauceDemo Login Page."""

    # UI Locators
    USERNAME_INPUT = (By.ID, "user-name")
    PASSWORD_INPUT = (By.ID, "password")
    LOGIN_BUTTON = (By.ID, "login-button")
    ERROR_CONTAINER = (By.CSS_SELECTOR, "h3[data-test='error']")
    LOGIN_CREDENTIALS_CONTAINER = (By.ID, "login_credentials")

    def __init__(self, driver: WebDriver):
        super().__init__(driver)

    def open(self) -> "LoginPage":
        """Navigates to the SauceDemo home / login page."""
        self.navigate_to(BASE_URL)
        return self

    def is_at_login_page(self) -> bool:
        """Verifies if the browser is currently at the login page."""
        return self.is_element_visible(self.LOGIN_BUTTON) and self.is_element_visible(self.USERNAME_INPUT)

    def enter_username(self, username: str) -> "LoginPage":
        """Enters the username into the username field."""
        self.enter_text(self.USERNAME_INPUT, username)
        return self

    def enter_password(self, password: str) -> "LoginPage":
        """Enters the password into the password field."""
        self.enter_text(self.PASSWORD_INPUT, password)
```

```

        return self

def click_login(self) -> None:
    """Submits the login credentials."""
    self.click_element(self.LOGIN_BUTTON)
    time.sleep(0.3)

def login(self, username: str = "", password: str = "") -> None:
    """Helper to fill credentials and click login."""
    self.enter_username(username)
    self.enter_password(password)
    self.click_login()

def login_as_standard_user(self) -> None:
    """Performs a valid login using the standard_user account."""
    self.open()
    self.login(VALID_USERNAME, VALID_PASSWORD)

def get_error_message(self) -> str:
    """Returns validation error banner text."""
    return self.get_element_text(self.ERROR_CONTAINER)

def is_error_displayed(self) -> bool:
    """Checks if the error banner is displayed."""
    return self.is_element_visible(self.ERROR_CONTAINER, timeout=2)

```

5.2 Inventory Page Object (automation_selenium/pages/inventory_page.py)

```
"""
InventoryPage module representing the product catalog page of SauceDemo.
Course: CST8513 - Quality Assurance and Testing (Assignment III)
Students: Devangbhai Pandit & Meet Ahalpara
"""

import time
from selenium.webdriver.common.by import By
from selenium.webdriver.remote.webdriver import WebDriver
from automation_selenium.pages.base_page import BasePage

class InventoryPage(BasePage):
    """Encapsulates locators and actions for the Product Inventory Page."""

    # UI Locators
    PAGE_TITLE = (By.CLASS_NAME, "title")
    INVENTORY_ITEMS = (By.CLASS_NAME, "inventory_item")
    ITEM_NAMES = (By.CLASS_NAME, "inventory_item_name")
    ITEM_PRICES = (By.CLASS_NAME, "inventory_item_price")
    SORT_DROPDOWN = (By.CLASS_NAME, "product_sort_container")
    ACTIVE_SORT_LABEL = (By.CLASS_NAME, "active_option")
    CART_ICON = (By.CLASS_NAME, "shopping_cart_link")
    CART_BADGE = (By.CLASS_NAME, "shopping_cart_badge")

    # Sidebar Navigation Locators
    MENU_BUTTON = (By.ID, "react-burger-menu-btn")
    LOGOUT_LINK = (By.ID, "logout_sidebar_link")

    def __init__(self, driver: WebDriver):
        super().__init__(driver)

    def is_at_inventory_page(self) -> bool:
        """Verifies if the user is on the inventory page."""
        return "inventory.html" in self.get_current_url() and self.is_element
_visible(self.PAGE_TITLE)

    def get_product_count(self) -> int:
        """Returns the total number of product items displayed."""
        return len(self.find_elements(self.INVENTORY_ITEMS))

    def get_all_product_names(self) -> list[str]:
        """Returns a list of all product names visible on the inventory grid.
        """
        return self.driver.execute_script(
            "return Array.from(document.querySelectorAll('.inventory_item_nam
e')).map(e => (e.textContent || '').trim());"
```

```

    )

    def get_all_product_prices(self) -> list[float]:
        """Returns a list of all product prices parsed as floats."""
        return self.driver.execute_script(
            "return Array.from(document.querySelectorAll('.inventory_item_pri
ce')).map(e => parseFloat((e.textContent || '').replace('$', '').trim()));"
        )

    def click_product_by_name(self, product_name: str) -> None:
        """Clicks a product's name link to navigate to its details page."""
        locator = (By.XPATH, f"//div[contains(@class, 'inventory_item_name')
and text()='{product_name}']")
        self.click_element(locator)
        time.sleep(0.3)

    def sort_by(self, option_value: str) -> None:
        """Sorts inventory: 'az', 'za', 'lohi', 'hilo'."""
        self.select_dropdown_by_value(self.SORT_DROPDOWN, option_value)
        time.sleep(0.4)

    def get_active_sort_text(self) -> str:
        """Returns currently active sort option label."""
        return self.driver.execute_script(
            "return (document.querySelector('.active_option')?.textContent ||
'').trim();"
        )

    def add_product_to_cart_by_name(self, product_name: str) -> None:
        """Adds a specific product to cart using its name."""
        locator = (By.XPATH, f"//div[contains(text(), '{product_name}')] / ances
tor::div[@class='inventory_item'] // button")
        self.click_element(locator)
        time.sleep(0.2)

    def remove_product_from_cart_by_name(self, product_name: str) -> None:
        """Removes a specific product from cart on inventory page."""
        locator = (By.XPATH, f"//div[contains(text(), '{product_name}')] / ances
tor::div[@class='inventory_item'] // button[contains(text(), 'Remove')]")
        self.click_element(locator)
        time.sleep(0.2)

    def get_cart_badge_count(self) -> int:
        """Returns integer count on shopping cart badge, or 0 if absent."""
        count_str = self.driver.execute_script(
            "return (document.querySelector('.shopping_cart_badge')?.textCont
ent || '').trim();"
        )
        return int(count_str) if count_str.isdigit() else 0

```

```
def click_cart_icon(self) -> None:
    """Clicks the shopping cart icon to open the cart page."""
    self.click_element(self.CART_ICON)
    time.sleep(0.3)

def click_logout(self) -> None:
    """Opens sidebar and clicks the Logout link."""
    self.click_element(self.MENU_BUTTON)
    time.sleep(0.5)
    self.click_element(self.LOGOUT_LINK)
    time.sleep(0.5)
```

6. Automated Test Suites (Assertions & Execution)

6.1 Authentication Test Suite

(automation_selenium/tests/test_authentication.py)

```
"""
Test Suite covering Authentication and Session Management (TC-01, TC-02, TC-1
4).
Course: CST8513 - Quality Assurance and Testing (Assignment III)
Students: Devangbhai Pandit & Meet Ahalpara
"""
```

```
import pytest
from automation_selenium.config.config import (
    BASE_URL,
    VALID_USERNAME,
    VALID_PASSWORD,
    LOCKED_USERNAME,
    INVALID_USERNAME,
    INVALID_PASSWORD,
)
from automation_selenium.pages.login_page import LoginPage
from automation_selenium.pages.inventory_page import InventoryPage

@pytest.mark.auth
@pytest.mark.regression
class TestAuthentication:
    """Test suite verifying user login, validation errors, and session termin
ation."""

    @pytest.mark.smoke
    def test_tc01_successful_login(self, driver):
        """TC-01 / FR-01: Verify successful login with valid credentials."""
        login_page = LoginPage(driver)
        login_page.open()
        login_page.login(VALID_USERNAME, VALID_PASSWORD)

        inventory_page = InventoryPage(driver)
        assert inventory_page.is_at_inventory_page(), "User should be redirec
ted to inventory.html"
        assert inventory_page.get_product_count() == 6, "Inventory should dis
play exactly 6 products"

    @pytest.mark.negative
    @pytest.mark.parametrize(
        "username, password, expected_error",
        [
            (INVALID_USERNAME, INVALID_PASSWORD, "Epic sadface: Username and
```

```

password do not match any user in this service"),
    (LOCKED_USERNAME, VALID_PASSWORD, "Epic sadface: Sorry, this user
has been locked out."),
    ("", VALID_PASSWORD, "Epic sadface: Username is required"),
    (VALID_USERNAME, "", "Epic sadface: Password is required"),
    ],
)
def test_tc02_login_error_validation(self, driver, username, password, expected_error):
    """TC-02 / FR-02: Verify error message validation for invalid and edge-case credentials."""
    login_page = LoginPage(driver)
    login_page.open()
    login_page.login(username, password)

    assert login_page.is_error_displayed(), "Error container should be visible"
    assert login_page.get_error_message() == expected_error, "Error message text must match"

@pytest.mark.smoke
def test_tc14_secure_logout(self, authenticated_driver):
    """TC-14 / FR-14: Verify secure user logout and session invalidation.

    inventory_page = InventoryPage(authenticated_driver)
    inventory_page.click_logout()

    login_page = LoginPage(authenticated_driver)
    assert login_page.is_at_login_page(), "User should be returned to login page"

    # Verify back navigation does not expose authenticated inventory
    authenticated_driver.get(f"{BASE_URL}inventory.html")
    assert login_page.is_error_displayed() or login_page.is_at_login_page
()

```

6.2 Checkout & Tax Calculations Suite

(automation_selenium/tests/test_checkout.py)

```
"""
```

Test Suite covering Checkout Flow, Field Validation, Calculations, and Order Completion (TC-09 to TC-13).

Course: CST8513 - Quality Assurance and Testing (Assignment III)

Students: Devangbhai Pandit & Meet Ahalpara

```
"""
```

```
import pytest
from automation_selenium.config.config import (
    DEFAULT_FIRST_NAME,
    DEFAULT_LAST_NAME,
    DEFAULT_POSTAL_CODE,
)
from automation_selenium.pages.inventory_page import InventoryPage
from automation_selenium.pages.cart_page import CartPage
from automation_selenium.pages.checkout_step_one_page import CheckoutStepOneP
age
from automation_selenium.pages.checkout_step_two_page import CheckoutStepTwoP
age
from automation_selenium.pages.checkout_complete_page import CheckoutComplete
Page

@pytest.mark.checkout
@pytest.mark.regression
class TestCheckout:
    """Test suite verifying checkout information validation, tax math, and or
der confirmation."""

    @pytest.mark.negative
    def test_tc09_checkout_empty_fields_validation(self, authenticated_drive
r):
        """TC-09 / FR-09: Verify required field validation when submitting bl
ank checkout form."""
        inventory_page = InventoryPage(authenticated_driver)
        inventory_page.add_product_to_cart_by_name("Sauce Labs Backpack")
        inventory_page.click_cart_icon()

        cart_page = CartPage(authenticated_driver)
        cart_page.click_checkout()

        step_one = CheckoutStepOnePage(authenticated_driver)
        step_one.click_continue()

        assert step_one.is_error_displayed()
        assert "Error: First Name is required" in step_one.get_error_message()
```

```

@pytest.mark.negative
@pytest.mark.parametrize(
    "first_name, last_name, postal_code, expected_error",
    [
        (DEFAULT_FIRST_NAME, "", DEFAULT_POSTAL_CODE, "Error: Last Name is required"),
        (DEFAULT_FIRST_NAME, DEFAULT_LAST_NAME, "", "Error: Postal Code is required"),
    ],
)
def test_tc10_checkout_input_validation(self, authenticated_driver, first_name, last_name, postal_code, expected_error):
    """TC-10 / FR-10: Verify validation for missing Last Name and Postal Code fields."""
    inventory_page = InventoryPage(authenticated_driver)
    inventory_page.add_product_to_cart_by_name("Sauce Labs Backpack")
    inventory_page.click_cart_icon()

    cart_page = CartPage(authenticated_driver)
    cart_page.click_checkout()

    step_one = CheckoutStepOnePage(authenticated_driver)
    step_one.fill_information(first_name, last_name, postal_code)
    step_one.click_continue()

    assert step_one.is_error_displayed()
    assert expected_error in step_one.get_error_message()

def test_tc11_order_overview_calculations(self, authenticated_driver):
    """TC-11 / FR-11: Verify item subtotal, 8% tax calculation, and total pricing on overview."""
    inventory_page = InventoryPage(authenticated_driver)
    inventory_page.add_product_to_cart_by_name("Sauce Labs Backpack")
    # $29.99
    inventory_page.add_product_to_cart_by_name("Sauce Labs Bolt T-Shirt")
    # $15.99
    inventory_page.click_cart_icon()

    cart_page = CartPage(authenticated_driver)
    cart_page.click_checkout()

    step_one = CheckoutStepOnePage(authenticated_driver)
    step_one.fill_information(DEFAULT_FIRST_NAME, DEFAULT_LAST_NAME, DEFAULT_POSTAL_CODE)
    step_one.click_continue()

    step_two = CheckoutStepTwoPage(authenticated_driver)
    assert step_two.is_at_step_two_page()

```

```

        subtotal = step_two.get_subtotal_amount()
        tax = step_two.get_tax_amount()
        total = step_two.get_total_amount()

        expected_subtotal = 45.98
        expected_tax = round(expected_subtotal * 0.08, 2)
        expected_total = round(expected_subtotal + expected_tax, 2)

        assert subtotal == expected_subtotal, f"Subtotal mismatch: got {subtotal}, expected {expected_subtotal}"
        assert tax == expected_tax, f"Tax mismatch: got {tax}, expected {expected_tax}"
        assert total == expected_total, f"Total mismatch: got {total}, expected {expected_total}"

    @pytest.mark.smoke
    def test_tc12_finish_order_confirmation(self, authenticated_driver):
        """TC-12 / FR-12: Verify successful order completion, confirmation text, and pony express graphic."""
        inventory_page = InventoryPage(authenticated_driver)
        inventory_page.add_product_to_cart_by_name("Sauce Labs Backpack")
        inventory_page.click_cart_icon()

        cart_page = CartPage(authenticated_driver)
        cart_page.click_checkout()

        step_one = CheckoutStepOnePage(authenticated_driver)
        step_one.fill_information(DEFAULT_FIRST_NAME, DEFAULT_LAST_NAME, DEFAULT_POSTAL_CODE)
        step_one.click_continue()

        step_two = CheckoutStepTwoPage(authenticated_driver)
        step_two.click_finish()

        complete_page = CheckoutCompletePage(authenticated_driver)
        assert complete_page.is_at_complete_page()
        assert complete_page.get_complete_header() == "Thank you for your order!"
        assert complete_page.is_pony_image_displayed()

```

7. Execution Commands & Technical References

7.1 Local Test Execution

- Run Headless (Fastest): `python -m pytest automation_selenium/tests/ -v`
- Run Headed (Visual Display): `python -m pytest automation_selenium/tests/ --headed -v`
- Run Unified Runner: `python run_tests.py --suite all`
- Run Robot Framework: `python -m robot --outputdir reports/robot_logs automation_robot/tests/saucedemo_suite.robot`

References

- [1] "WebDriver," Selenium, [Online]. Available: <https://www.selenium.dev/documentation/webdriver/>.
- [2] "pytest: helps you write better programs," [Online]. Available: <https://docs.pytest.org/en/stable/>.
- [3] "SeleniumLibrary," RobotFramework, [Online]. Available: <https://robotframework.org/SeleniumLibrary/>.
- [4] "SauceDemo," [Online]. Available: <https://www.saucedemo.com/>.
- [5] "GitHub Repo," [Online]. Available: <https://github.com/MeetAhalpara/QA-Test-Automation>.